

Universidade de Brasília
Faculdade de Tecnologia

Cálculo Numérico Aplicado

Programa para Casa #4 (PPC4)

Prof. Dr. Rafael Gabler Gontijo

Mateus Leal Silva

Matrícula: 221028134

Brasília, 03 de junho de 2026

Sumário

1	Introdução	2
2	Formulação Matemática do Problema	2
3	Linha de Busca por Interpolação Quadrática	3
4	Método do Aclive Máximo	3
5	Método dos Gradientes Conjugados de Fletcher–Reeves	4
6	Arquivos de Saída do Programa	4
7	Resultados Numéricos para o Ponto Inicial $(-2, 3)$	4
7.1	Aclive Máximo	5
7.2	Gradientes Conjugados de Fletcher–Reeves	5
8	Código-Fonte Python	5
9	Conclusão	9

1 Introdução

Este relatório apresenta o desenvolvimento de um programa computacional para resolver um problema de maximização bidimensional sem restrições por meio de dois métodos clássicos baseados em gradientes: o **método do alicive máximo** e o **método dos gradientes conjugados na variante de Fletcher–Reeves**.

O problema proposto consiste em maximizar a função objetivo

$$f(x, y) = 2xy + 2x - x^2 - 2y^2, \quad (1)$$

cujo ponto ótimo analítico é

$$(x^*, y^*) = (2, 1), \quad f^* = 2. \quad (2)$$

Além de implementar os dois métodos, o programa deve executar a linha de busca unidimensional por interpolação quadrática de três pontos, salvar os logs de cada método em arquivos separados e gerar uma malha de amostras da função objetivo para posterior construção de curvas de nível.

2 Formulação Matemática do Problema

A função objetivo é uma função quadrática de duas variáveis. O seu gradiente é dado por

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2y + 2 - 2x \\ 2x - 4y \end{bmatrix}. \quad (3)$$

A matriz Hessiana associada à função é

$$H_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} -2 & 2 \\ 2 & -4 \end{bmatrix}. \quad (4)$$

O ponto crítico é obtido impondo

$$\nabla f(x, y) = 0. \quad (5)$$

Assim,

$$2y + 2 - 2x = 0 \quad \Rightarrow \quad x = y + 1, \quad (6)$$

$$2x - 4y = 0 \quad \Rightarrow \quad x = 2y. \quad (7)$$

Substituindo uma relação na outra, obtém-se

$$2y = y + 1 \quad \Rightarrow \quad y = 1, \quad (8)$$

e, conseqüentemente,

$$x = 2. \quad (9)$$

Portanto, o ponto crítico da função objetivo é

$$(x^*, y^*) = (2, 1). \quad (10)$$

Como a Hessiana possui autovalores negativos,

$$\lambda_1 = -3 + \sqrt{5} < 0, \quad \lambda_2 = -3 - \sqrt{5} < 0, \quad (11)$$

a matriz Hessiana é definida negativa. Logo, o ponto crítico corresponde a um ponto de máximo da função.

3 Linha de Busca por Interpolação Quadrática

Em ambos os métodos, uma vez definida a direção de busca p_k , o próximo ponto é obtido por

$$z_{k+1} = z_k + h_k p_k, \quad z_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}. \quad (12)$$

O valor do passo h_k é calculado por uma busca unidimensional aplicada à função

$$\phi(h) = f(z_k + h p_k). \quad (13)$$

A interpolação quadrática utiliza três pontos distintos h_0 , h_1 e h_2 , com valores correspondentes $\phi(h_0)$, $\phi(h_1)$ e $\phi(h_2)$. O vértice da parábola interpoladora é dado por

$$h^* = \frac{\phi(h_0)(h_1^2 - h_2^2) + \phi(h_1)(h_2^2 - h_0^2) + \phi(h_2)(h_0^2 - h_1^2)}{2\phi(h_0)(h_1 - h_2) + 2\phi(h_1)(h_2 - h_0) + 2\phi(h_2)(h_0 - h_1)}. \quad (14)$$

Na implementação, adotou-se

$$h_0 = 0, \quad h_1 = 1, \quad h_2 = 2. \quad (15)$$

Como a função objetivo é quadrática, essa interpolação recupera exatamente o máximo da função $\phi(h)$ ao longo da direção de busca, salvo em situações degeneradas. Caso o denominador da Equação (14) fique muito pequeno, foi implementado um fallback baseado na curvatura local:

$$h^* = -\frac{\nabla f(z_k)^T p_k}{p_k^T H_f p_k}. \quad (16)$$

4 Método do Aclive Máximo

No método do aclive máximo, a direção de busca é sempre definida pelo gradiente local da função objetivo:

$$p_k = \nabla f(z_k). \quad (17)$$

Assim, a cada iteração, o algoritmo executa os seguintes passos:

1. calcula o gradiente $\nabla f(z_k)$;
2. define $p_k = \nabla f(z_k)$;
3. calcula h_k por interpolação quadrática;
4. atualiza o ponto por $z_{k+1} = z_k + h_k p_k$;
5. calcula o erro $\|\nabla f(z_k)\|_2$ e verifica a convergência.

Esse método é simples e robusto, mas pode apresentar trajetória em zigue-zague em funções quadráticas alongadas, especialmente quando as curvas de nível apresentam forte anisotropia.

5 Método dos Gradientes Conjugados de Fletcher–Reeves

O método dos gradientes conjugados busca melhorar a eficiência do aclave máximo usando direções de busca que incorporam informação das iterações anteriores.

Para o problema de maximização, a direção inicial é

$$p_0 = \nabla f(z_0). \quad (18)$$

Após cada avanço, calcula-se o novo gradiente e o parâmetro de Fletcher–Reeves:

$$\beta_k = \frac{\|\nabla f(z_{k+1})\|_2^2}{\|\nabla f(z_k)\|_2^2}. \quad (19)$$

A nova direção de busca é atualizada por

$$p_{k+1} = \nabla f(z_{k+1}) + \beta_k p_k. \quad (20)$$

Para uma função quadrática e aritmética exata, o método de gradientes conjugados tende a convergir em, no máximo, n iterações, em que n é a dimensão do problema. Como este problema possui duas variáveis, espera-se convergência em aproximadamente duas iterações quando a busca linear é suficientemente precisa.

6 Arquivos de Saída do Programa

O programa gera três arquivos de saída, conforme especificado no enunciado:

Tabela 1: Arquivos gerados pelo programa.

Arquivo	Método	Conteúdo
output1.dat	Aclave máximo	Log iterativo do primeiro método
output2.dat	Fletcher–Reeves	Log iterativo do segundo método
function.dat	Malha da função	Amostras (x, y, f) para curvas de nível

Cada linha dos arquivos `output1.dat` e `output2.dat` possui o seguinte formato:

$$\text{iter erro h x y dfx dfy}. \quad (21)$$

O erro adotado no programa é a norma euclidiana do gradiente:

$$\text{erro}_k = \|\nabla f(z_k)\|_2. \quad (22)$$

7 Resultados Numéricos para o Ponto Inicial $(-2, 3)$

Para validar o programa, foi utilizada a entrada padrão

$$(x_0, y_0) = (-2, 3). \quad (23)$$

7.1 Aclive Máximo

A Tabela 2 apresenta as primeiras iterações obtidas pelo método do aclive máximo. Os valores de x , y , $\partial f/\partial x$ e $\partial f/\partial y$ correspondem ao ponto antes da atualização de cada iteração.

Tabela 2: Iterações iniciais do método do aclive máximo.

k	erro	h	x	y	$\partial f/\partial x$	$\partial f/\partial y$
0	20.000000	0.192308	-2.000000	3.000000	12.000000	-16.000000
1	1.538462	1.250000	0.307692	-0.076923	1.230769	0.923077
2	0.769231	0.192308	1.846154	1.076923	0.461538	-0.615385
3	0.059172	1.250000	1.934911	0.958580	0.047337	0.035503
4	0.029586	0.192308	1.994083	1.002959	0.017751	-0.023669
5	0.002276	1.250000	1.997497	0.998407	0.001821	0.001365

A trajetória do aclive máximo se aproxima progressivamente do ponto ótimo. Após as iterações executadas até a tolerância numérica adotada, o programa retorna

$$(x, y) \approx (1.999999991236, 0.999999997696), \quad (24)$$

com

$$f(x, y) \approx 2.000000000000. \quad (25)$$

7.2 Gradientes Conjugados de Fletcher–Reeves

A Tabela 3 mostra o comportamento do método de Fletcher–Reeves para o mesmo ponto inicial.

Tabela 3: Iterações do método de Fletcher–Reeves.

k	erro	h	x	y	$\partial f/\partial x$	$\partial f/\partial y$
0	20.000000	0.192308	-2.000000	3.000000	12.000000	-16.000000
1	1.538462	1.300000	0.307692	-0.076923	1.230769	0.923077
2	8.1886×10^{-15}	0.000000	2.000000	1.000000	5.3291×10^{-15}	-6.2172×10^{-15}

Como esperado para uma função quadrática em duas variáveis com linha de busca precisa, o método de Fletcher–Reeves alcança o ponto ótimo em duas atualizações efetivas:

$$(x, y) = (2, 1), \quad f(x, y) = 2. \quad (26)$$

8 Código-Fonte Python

A implementação completa do programa é apresentada a seguir. O código solicita ao usuário os valores iniciais (x_0, y_0) , executa os dois métodos e salva os arquivos `output1.dat`, `output2.dat` e `function.dat`.

```

1 import numpy as np
2
3 # =====
4 # PPC4 - Calculo Numerico Aplicado
5 # Maximizacao de f(x,y) por:
6 # 1) Aclive maximo
7 # 2) Gradientes conjugados de Fletcher-Reeves
8 # =====
9
10 HESSIAN = np.array([[ -2.0, 2.0],
11                     [ 2.0, -4.0]])
12
13
14 def funcao(x, y):
15     """Funcao objetivo: f(x,y) = 2xy + 2x - x^2 - 2y^2."""
16     return 2.0*x*y + 2.0*x - x**2 - 2.0*y**2
17
18
19 def gradiente(x, y):
20     """Gradiente de f: [df/dx, df/dy]."""
21     dfdx = 2.0*y + 2.0 - 2.0*x
22     dfdy = 2.0*x - 4.0*y
23     return np.array([dfdx, dfdy], dtype=float)
24
25
26 def norma(v):
27     return float(np.sqrt(np.dot(v, v)))
28
29
30 def interpolacao_quadratica(z, p, passo_base=1.0, eps=1.0e-14):
31     """
32     Linha de busca unidimensional por interpolacao quadratica.
33     Ajusta uma parabola em h = 0, h = passo_base e h = 2*passo_base
34     para phi(h) = f(z + h*p) e retorna o vertice da parabola.
35
36     Caso o denominador do vertice fique muito pequeno, usa fallback
37     baseado na curvatura quadratica local da propria funcao.
38     """
39     h0 = 0.0
40     h1 = passo_base
41     h2 = 2.0*passo_base
42
43     def phi(h):
44         ponto = z + h*p
45         return funcao(ponto[0], ponto[1])
46
47     f0 = phi(h0)
48     f1 = phi(h1)
49     f2 = phi(h2)
50
51     numerador = (f0*(h1**2 - h2**2)
52                 + f1*(h2**2 - h0**2)
53                 + f2*(h0**2 - h1**2))
54     denominador = (2.0*f0*(h1 - h2)
55                   + 2.0*f1*(h2 - h0)
56                   + 2.0*f2*(h0 - h1))
57
58     if abs(denominador) > eps:

```

```

59         return numerador/denominador
60
61     # Fallback: para funcao quadratica,  $\phi'(h) = \text{grad}^T p + h p^T H p$ .
62     g = gradiente(z[0], z[1])
63     curvatura = float(np.dot(p, HESSIAN @ p))
64     inclinacao = float(np.dot(g, p))
65     if abs(curvatura) < eps:
66         return 0.0
67     return -inclinacao/curvatura
68
69
70 def escrever_log(nome_arquivo, linhas):
71     """Grava o log no formato: iter erro h x y dfx dfy."""
72     with open(nome_arquivo, "w", encoding="utf-8") as arq:
73         arq.write("# iter erro h x y dfx dfy\n")
74         for linha in linhas:
75             k, erro, h, x, y, dfx, dfy = linha
76             arq.write(f"{k:4d} {erro: .12e} {h: .12e} "
77                     f"{x: .12e} {y: .12e} {dfx: .12e} {dfy: .12e}\n")
78
79
80 def aclone_maximo(z0, tolerancia=1.0e-8, max_iter=100):
81     """Metodo do aclone maximo usando  $p_k = \text{grad } f_k$ ."""
82     z = np.array(z0, dtype=float)
83     log = []
84
85     for k in range(max_iter + 1):
86         g = gradiente(z[0], z[1])
87         erro = norma(g)
88
89         if erro < tolerancia:
90             log.append((k, erro, 0.0, z[0], z[1], g[0], g[1]))
91             break
92
93         p = g.copy()
94         h = interpolacao_quadratica(z, p)
95         log.append((k, erro, h, z[0], z[1], g[0], g[1]))
96         z = z + h*p
97
98     return z, log
99
100
101 def fletcher_reeves(z0, tolerancia=1.0e-8, max_iter=100):
102     """Metodo dos gradientes conjugados na variante de Fletcher-Reeves."""
103
104     z = np.array(z0, dtype=float)
105     g = gradiente(z[0], z[1])
106     p = g.copy()
107     log = []
108
109     for k in range(max_iter + 1):
110         erro = norma(g)
111
112         if erro < tolerancia:
113             log.append((k, erro, 0.0, z[0], z[1], g[0], g[1]))
114             break
115
116         h = interpolacao_quadratica(z, p)

```



```

116         log.append((k, erro, h, z[0], z[1], g[0], g[1]))
117
118         z_novo = z + h*p
119         g_novo = gradiente(z_novo[0], z_novo[1])
120
121         denom = float(np.dot(g, g))
122         beta = float(np.dot(g_novo, g_novo)/denom) if denom > 0.0 else
0.0
123         p = g_novo + beta*p
124
125         z = z_novo
126         g = g_novo
127
128     return z, log
129
130
131 def gerar_function_dat(nome_arquivo="function.dat", xmin=-4.0, xmax=4.0,
132                       ymin=-3.0, ymax=5.0, nx=101, ny=101):
133     """Gera amostras x, y, f(x,y) para curvas de nivel."""
134     xs = np.linspace(xmin, xmax, nx)
135     ys = np.linspace(ymin, ymax, ny)
136
137     with open(nome_arquivo, "w", encoding="utf-8") as arq:
138         arq.write("# x y f\n")
139         for x in xs:
140             for y in ys:
141                 arq.write(f"{x: .12e} {y: .12e} {funcao(x, y): .12e}\n")
142             arq.write("\n")
143
144
145 def ler_float(mensagem, padrao):
146     texto = input(mensagem).strip().replace(",", ".")
147     if texto == "":
148         return float(padrao)
149     return float(texto)
150
151
152 def main():
153     print("PPC4 - Otimizacao bidimensional")
154     print("Funcao: f(x,y) = 2xy + 2x - x^2 - 2y^2")
155     print("Pressione ENTER para usar o ponto inicial padrao (-2, 3).")
156
157     x0 = ler_float("x0 = ", -2.0)
158     y0 = ler_float("y0 = ", 3.0)
159
160     z0 = np.array([x0, y0], dtype=float)
161
162     sol1, log1 = aclave_maximo(z0)
163     sol2, log2 = fletcher_reeves(z0)
164
165     escrever_log("output1.dat", log1)
166     escrever_log("output2.dat", log2)
167     gerar_function_dat("function.dat")
168
169     print("\nArquivos gerados:")
170     print("  output1.dat  - Aclave maximo")
171     print("  output2.dat  - Fletcher-Reeves")
172     print("  function.dat - Amostras para curvas de nivel")

```

```

173
174     print("\nResultado - Aclive maximo:")
175     print(f"    x = {sol1[0]:.12f}, y = {sol1[1]:.12f}, f = {funcao(sol1
176           [0], sol1[1]):.12f}")
177
178     print("\nResultado - Fletcher-Reeves:")
179     print(f"    x = {sol2[0]:.12f}, y = {sol2[1]:.12f}, f = {funcao(sol2
180           [0], sol2[1]):.12f}")
181
182     print("\n0timo analitico esperado:")
183     print("    x* = 2, y* = 1, f* = 2")
184
185 if __name__ == "__main__":
    main()

```

Listing 1: Programa completo para o PPC4.

9 Conclusão

O programa desenvolvido atende aos requisitos propostos para o PPC4, pois implementa duas estratégias de maximização bidimensional baseadas em gradientes, utiliza a mesma linha de busca unidimensional por interpolação quadrática e salva os resultados nos arquivos especificados.

O método do aclave máximo apresentou convergência gradual para o ponto ótimo, com trajetória mais lenta e sujeita ao comportamento em zigue-zague. Já o método dos gradientes conjugados de Fletcher-Reeves convergiu de forma muito mais rápida, alcançando o ótimo analítico em duas atualizações efetivas para o ponto inicial testado.

Os resultados numéricos obtidos confirmam o ótimo analítico da função:

$$(x^*, y^*) = (2, 1), \quad f^* = 2. \quad (27)$$

Referências

- [1] CHAPRA, S. C.; CANALE, R. P. **Métodos Numéricos para Engenharia**. McGrawHill, 5^a edição, 2008.
- [2] GONTIJO, R. G. **Notas de aula do curso de Cálculo Numérico Aplicado**. Universidade de Brasília (UnB), 2026.